

Exception Handling in Java

Important Exception Examples with Outputs

Package: exceptionhandling

This PDF contains the important exception-handling examples from ArithmeticException through User-Defined Exception. Each program uses the package name **exceptionhandling** and includes the expected output.

1. ArithmeticException

Explanation: Occurs when an invalid arithmetic operation is performed, commonly division by zero.

Sample Program

```
package exceptionhandling;

public class ArithmeticDemo {
    public static void main(String[] args) {
        try {
            int a = 10;
            int b = 0;
            int result = a / b;
            System.out.println(result);
        }
        catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

Output

```
Cannot divide by zero
```

2. ArrayIndexOutOfBoundsException

Explanation: Occurs when an invalid array index is accessed. For three elements, valid indexes are 0, 1 and 2.

Sample Program

```
package exceptionhandling;

public class ArrayDemo {
    public static void main(String[] args) {
        try {
            int[] numbers = {10, 20, 30};
            System.out.println(numbers[5]);
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index");
        }
    }
}
```

Output

```
Invalid array index
```

3. StringIndexOutOfBoundsException

Explanation: Occurs when an invalid character position is accessed in a String.

Sample Program

```
package exceptionhandling;

public class StringDemo {
    public static void main(String[] args) {
        try {
            String str = "Java";
            System.out.println(str.charAt(10));
        }
        catch (StringIndexOutOfBoundsException e) {
            System.out.println("Invalid string index");
        }
    }
}
```

Output

```
Invalid string index
```

4. NullPointerException

Explanation: Occurs when a method or property is accessed through a null reference.

Sample Program

```
package exceptionhandling;

public class NullDemo {
    public static void main(String[] args) {
        try {
            String str = null;
            System.out.println(str.length());
        }
        catch (NullPointerException e) {
            System.out.println("String is null");
        }
    }
}
```

Output

```
String is null
```

5. NumberFormatException

Explanation: Occurs when an invalid String is converted into a number.

Sample Program

```
package exceptionhandling;

public class NumberDemo {
    public static void main(String[] args) {
        try {
            String str = "abc";
            int number = Integer.parseInt(str);
            System.out.println(number);
        }
        catch (NumberFormatException e) {
            System.out.println("Invalid number format");
        }
    }
}
```

Output

```
Invalid number format
```

6. ClassCastException

Explanation: Occurs when an object is incorrectly cast from one type to another.

Sample Program

```
package exceptionhandling;

public class CastDemo {
    public static void main(String[] args) {
        try {
            Object obj = "Hello";
            Integer number = (Integer) obj;
            System.out.println(number);
        }
        catch (ClassCastException e) {
            System.out.println("Invalid type casting");
        }
    }
}
```

Output

```
Invalid type casting
```

7. InputMismatchException

Explanation: Occurs when Scanner receives input of a different type from the type requested.

Sample Program

```
package exceptionhandling;

import java.util.InputMismatchException;
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.println("Enter your age:");
            int age = sc.nextInt();
            System.out.println("Age: " + age);
        }
        catch (InputMismatchException e) {
            System.out.println("Please enter a valid number");
        }
    }
}
```

Output

```
Enter your age:
abc
Please enter a valid number
```

8. FileNotFoundException

Explanation: Occurs when Java tries to open a file that does not exist.

Sample Program

```
package exceptionhandling;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileDemo {
    public static void main(String[] args) {
        try {
            File file = new File("student.txt");
            Scanner sc = new Scanner(file);
            System.out.println(sc.nextLine());
            sc.close();
        }
        catch (FileNotFoundException e) {
            System.out.println("File not found");
        }
    }
}
```

Output

```
File not found
```

9. IOException

Explanation: IOException is commonly used for input/output operations such as reading files.

Sample Program

```
package exceptionhandling;

import java.io.FileReader;
import java.io.IOException;

public class IODemo {
    public static void main(String[] args) {
        try {
            FileReader file = new FileReader("student.txt");
            int data = file.read();
            System.out.println((char) data);
            file.close();
        }
        catch (IOException e) {
            System.out.println("Input or output error");
        }
    }
}
```

Output

```
Input or output error
```

10. User-Defined Exception

Explanation: A custom exception is created by extending the Exception class and is thrown when the age is below 18.

Sample Program

```
package exceptionhandling;

class AgeException extends Exception {
    AgeException(String message) {
        super(message);
    }
}

public class UserDefinedDemo {

    static void checkAge(int age) throws AgeException {
        if (age < 18) {
            throw new AgeException("Age must be 18 or above");
        }

        System.out.println("Eligible to vote");
    }

    public static void main(String[] args) {
        try {
            checkAge(15);
        }
        catch (AgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output

```
Age must be 18 or above
```

Key Exception Handling Keywords

Keyword	Purpose
try	Contains code that may cause an exception.
catch	Handles an exception.
finally	Executes cleanup code after try/catch processing.
throw	Explicitly throws an exception.
throws	Declares that a method may throw an exception.